

Article

LeONet: A Hybrid Deep Learning Approach for High-Precision Code Clone Detection Using Abstract Syntax Tree Features

Thanoshan Vijayanandan ¹, Kuhaneswaran Banujan ², Ashan Induranga ^{1,3}, Banage T. G. S. Kumara ^{1,4,*}
and Kaveenga Koswattage ^{1,3,*}

¹ Center for Nano Device Fabrication and Characterization (CNFC), Faculty of Technology, Sabaragamuwa University of Sri Lanka, Belihuloya 70140, Sri Lanka; mvthanoshan@gmail.com (T.V.); ashaninduranga@tech.sab.ac.lk (A.I.)

² Faculty of Science and Engineering, Southern Cross University, Lismore, NSW 2480, Australia; bhakuha@gmail.com

³ Department of Engineering Technology, Faculty of Technology, Sabaragamuwa University of Sri Lanka, Belihuloya 70140, Sri Lanka

⁴ Department of Data Science, Faculty of Computing, Sabaragamuwa University of Sri Lanka, Belihuloya 70140, Sri Lanka

* Correspondence: koswattagekr@appsc.sab.ac.lk (B.T.G.S.K.); kumara@appsc.sab.ac.lk (K.K.)

Abstract

Code duplication, commonly referred to as code cloning, is not inherent in software systems but arises due to various factors, such as time constraints in meeting project deadlines. These duplications, or “code clones”, complicate the program structure and increase maintenance costs. Code clones are categorized into four types: Type-1, Type-2, Type-3, and Type-4. This study aims to address the adverse effects of code clones by introducing LeONet, a hybrid Deep Learning approach that enhances the detection of code clones in software systems. The hybrid approach, LeONet, combines LeNet-5 with Oreos Siamese architecture. We extracted clone method pairs from the BigCloneBench Java repository. Feature extraction was performed using Abstract Syntax Trees, which are scalable and accurately represent the syntactic structure of the source code. The performance of LeONet was compared against other classifiers including ANN, LeNet-5, Oreos Siamese, Light-GBM, XGBoost, and Decision Tree. LeONet demonstrated superior performance among the classifiers tested, achieving the highest F1 score of 98.12%. It also compared favorably against state-of-the-art approaches, indicating its effectiveness in code clone detection. The results validate the effectiveness of LeONet in detecting code clones, outperforming existing classifiers and competing closely with advanced methods. This study underscores the potential of hybrid deep learning models and feature extraction techniques in improving the accuracy of code clone detection, providing a promising direction for future research in this area.

Keywords: code cloning; software engineering; deep learning; Abstract Syntax Trees



Academic Editor: Domenico Ursino

Received: 31 May 2025

Revised: 29 June 2025

Accepted: 11 July 2025

Published: 15 July 2025

Citation: Vijayanandan, T.; Banujan, K.; Induranga, A.; Kumara, B.T.G.S.; Koswattage, K. LeONet: A Hybrid Deep Learning Approach for High-Precision Code Clone Detection Using Abstract Syntax Tree Features. *Big Data Cogn. Comput.* **2025**, *9*, 187. <https://doi.org/10.3390/bdcc9070187>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Developers tend to copy and paste code when building software, either with or without alterations [1]. Code clone is the term for this copied or duplicated code. The practice of duplicating source code is called code cloning [2,3]. Software systems do not naturally experience code clones. Several variables could compel or persuade software developers to create duplicate code, for example: various reuse and programming approaches in software

development strategy, such as reusing code through direct copy and paste, and generating code using tools that could result in code clones; added advantages in maintenance; limitations of developers; limitations in programming languages, such as the lack of reuse mechanisms; and duplicating by accident [4].

Various researchers have explored the serious negative impacts of code clones. They are considered bad coding practice and complicate the maintainability of software [5,6]. Code clones can be a big problem during development and maintenance, since inconsistent clones are a primary source of problems [7]. Code clones facing late propagations impact fault proneness [8]. Code clones pose a higher risk of instability than non-cloned code [9]. Over 18.42% of clone fragments that face bug fixes contain propagated bugs. Compared to code clones in different files, code clones in the same file have a larger chance of including propagated defects, and therefore, code cloning can sometimes be used to spread significant problems [10]. Code cloning enables attackers to inject harmful code [11]. Vulnerability propagation could happen if a vulnerable code segment is cloned [12].

There are four types of code clones: Type-1 (T1), Type-2 (T2), Type-3 (T3), and Type-4 (T4) [4,13]. T1 clones differ only in layouts, comments, and whitespace but share the same syntactic structure. T2 clones are code fragments that differ only in literals, identifiers, and data types in addition to T1 differences but are otherwise syntactically identical. T3 clones are syntactically similar (albeit not entirely identical) code fragments, besides differences in statements (statements can be added, updated, or removed) and T1 and T2 differences. Syntactically dissimilar code fragments that carry out the same computation are known as T4 clones [4,13].

The adverse effects of code clones in software have made code clone detection (CCD) a prominent research area [2]. CCD techniques can be categorized into traditional and machine learning (ML) [14]. This paper used ML and deep learning (DL) approaches to model CCD.

Definition 1 (Method). Method M indicates a Java method. An ordered list of statements is enclosed in two curly brackets, S_i , $i = 1, \dots, N$ illustrates the intended behaviour of the method, such as assignments, method calls, loops, and branching.

$$M = \langle S_1, \dots, S_N \rangle \quad (1)$$

Definition 2 (Code Clone). Methods M_i and M_j are considered as a code clone pair, if they score above a specific threshold according to a predefined similarity criterion [15]. Equation (2) is retrieved from [15].

$$\text{clone}(M_i, M_j) = \begin{cases} 1, & \text{if } \text{sim}(M_i, M_j) > \theta \\ 0, & \text{otherwise.} \end{cases} \quad (2)$$

In Figure 1, the original method and its T1 clone method become syntactically identical after trimming the whitespaces, layouts, and comments. After trimming and normalizing identifiers, literals, and types, the original method and its T2 clone method become syntactically identical. The actual method is syntactically like its T3 clone method after trimming and normalizing, even if a new statement is added and an existing statement is modified. The T4 clone method differs syntactically from the original method. They are very different in shape, but they all do the same computation. When focusing on the original method, for instance, it utilizes a for loop to calculate the sum value of the first “n” natural numbers, but the T4 clone method employs recursion to do the same. Hence, both are identical in semantics, functionality, and computation.

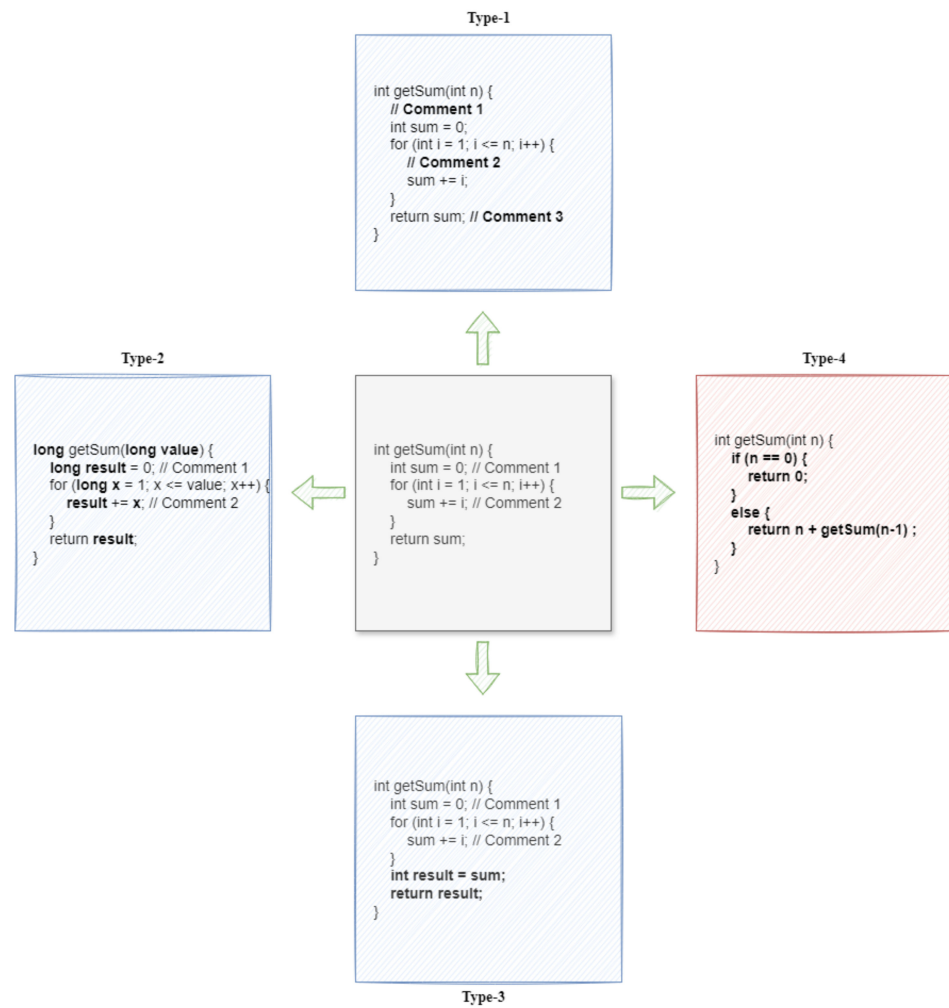


Figure 1. Example of different types of code clones.

Abstract Syntax Trees (ASTs) accurately capture the syntactic structure of source code and are scalable with large amounts of code, so it makes sense to extract features using them [16,17]. In this study, 48 features were extracted from ASTs, including 20 features that were not considered in previous studies. Several CCD ML and DL models were developed, including LeNet-5 [18] and Oreo's Siamese architecture [19]. LeONet, the proposed hybrid DL approach, is introduced to learn more from the given input and to improve the CCD performance.

The rest of this paper is structured as follows: The literature review is discussed in Section 2. Section 3 introduces the methodology. Section 4 contains the results and discussion. Limitations of this study are discussed in Section 5. Finally, this paper concludes with a conclusion and future work in Section 6.

2. Literature Review

In the past 25 years, the detection of clones in software systems has attracted considerable attention from researchers [20]. The term code clone has no precise definition. Different researchers refer to code cloning in a variety of ways. Code fragments identical to another code fragment, similar code, and duplicated code are some definitions coined in various research [2]. Two types of CCD methods exist: traditional and ML [14]. Traditional CCD approaches are hybrid, text-based, token-based, tree-based, Program Dependency Graph (PDG)-based, and metrics-based [3,4,14].

Target source code is perceived as a collection of strings in a text-based approach. Code clones are found by assessing the string sequences of two different code segments. The token-based approach parses the target source code into a series of tokens. The series is scanned to look for duplicated tokens [4]. Target source code is parsed into an AST in the tree-based approach. Related subtrees are looked for in the parsed tree using tree-matching methods. Actual code fragments corresponding to related subtrees are then returned as code clones. An approach that relies on PDG parses the target source code into PDGs. The isomorphic subgraph matching algorithm is then used to find related subgraphs. Actual code fragments corresponding to related subgraphs are returned as code clones [4].

Different metrics are extracted from code fragments using the metrics-based technique. Vectors of metrics are compared to find code clones. Hybrid code representation and/or the combined use of text-based, token-based, tree-based, PDG-based, and metrics-based techniques are categorized as hybrid approaches. However, these hybrid approaches can still be categorized under the previously mentioned text-based, token-based, tree-based, PDG-based, and metrics-based techniques [4].

ML makes machines more automatic, enabling them to learn to detect patterns, including relationships, trends, and underlying structures in the data, and make critical decisions. The rapid growth and widespread use of DL approaches can be credited to their capacity to discover characteristics in complicated, highly dimensional data with excellent predicted accuracy [21]. Due to the effectiveness of ML and DL approaches, they are utilized for CCD [14].

DL techniques have gained popularity in detecting code clones recently, along with ML techniques. Feature extraction is crucial for detecting code clones in both ML and DL approaches. Recent studies focused on extracting features from AST, PDG, and Bytecode Dependency Graph (BDG) representations of source code; method-level software metrics; and source code attributes related to data and control flow graphs. Precision, recall, and F1 score are the widely used evaluation metrics when evaluating the performance of models. Subject systems written in Java are widely used in ML-based CCD. Among publicly available Java clone benchmarks, BigCloneBench is the most widely used [20].

Sajnani et al. [22] developed a token-based code clone detector. It parses code blocks from source files and tokenizes them using a scanner that understands the token and block semantics of the language. Then, it builds an inverted index mapping token to their respective blocks. Unlike previous methods, it employs a filtering heuristic to create a partial index of only a subset of tokens in each block, rather than indexing all tokens. It processes all code blocks to find candidate clone blocks from the index. It applies a filtering heuristic to assess upper and lower similarity bounds between the query and candidate blocks. Candidates are accepted if their lower-bound similarity exceeds the threshold. It was able to detect T1, T2, and T3 code clones. IJaDataset, BigCloneBench, and Mutation Framework were utilized.

White et al. [23] proposed a learning-based CCD method. Their learning-centered approach was twofold: RNN and Recursive Neural Network (RvNN). RNN was a certain kind of language model that mapped each phrase in a fragment to an embedding. RvNN encoded embeddings of arbitrary length to characterize fragments. Ultimately, the source code was represented as encoded fragments. Their subject systems were ANTLR 4, Apache Ant 1.9.6, ArgoUML 0.34, CAROL 2.0.5, dnsjava 2.0.0, Hibernate 2, JDK 1.4.2, and JHotDraw 6. They constructed the AST using a fragment, greedily encoding nodes until they arrived at a code for a node encompassing the fragment. The code was compared to other greedily encoded fragments to find clones using a threshold. They detected entire clone categories.

Jiang et al. [24] developed an approach that can effectively find subtrees (smaller tree structures within a larger tree) that are similar to each other and demonstrated its

application to tree representations of source code. Initially, parse trees were built from the source code. A collection of vectors was created while preserving important details of the parse trees. These vectors were clustered based on Euclidean distance to detect code clones. JDK 1.4.2 and Linux kernel 2.6.16 were utilized for experimentation purposes.

Zhao and Huang [25] measured functional similarity of the code using DL to detect code clones. First, the data flow graph and control flow graph were built and encoded into a feature matrix. A feed-forward neural network extracted high-level features from the feature matrix. These high-level features were entered into a binary classification module to verify whether a pair of code fragments was functionally similar. The feature matrix was the source code representation of this approach. The DL approach was applied to determine code clones. Their approach was able to detect whole code clone types. Google Code Jam and BigCloneBench datasets were utilized.

Wu et al. [26] blended the accuracy of graph-based methods for CCD with the versatility of token-based methods. Through static code analysis, they generated CFG and applied centrality analysis to extract the centrality of each basic CFG block. These centrality tokens were computed, and semantic tokens were created. Tokens were vectorized and input into the Siamese network for CCD. Ultimately, the source code was represented as vectors. Siamese neural network architecture was used for CCD. They were able to classify all kinds of clones. Google Code Jam and BigCloneBench datasets were utilized.

Zhang et al. [27] suggested an innovative source code representation technique and employed it to categorize code clones. They parsed the code into an AST and divided it into a series of statement trees with the help of a preorder traversal algorithm. Then, vectors were created by encoding this statement of trees. These vectors were combined into a single vector with the help of the Bidirectional Gated Recurrent Unit model, which was their source code representation technique. Code segments that require clone detection were preprocessed into a sequence of statements of the tree and fed into their trained model. All kinds of clones were detected with this approach. BigCloneBench was utilized in the study.

Hu et al. [28] proposed a scalable tree-based CCD technique. They first extracted AST and converted it into a tree graph representation. Then, with the help of centrality analysis, they created a 72-dimensional vector while preserving AST details. Eventually, the vector was input into the CCD ML model, including 1-Nearest Neighbor, 3-Nearest Neighbor, Decision Tree, Random Forest, and Logistic Regression. All kinds of clones were classified according to their approach. Google Code Jam and BigCloneBench datasets were utilized.

To detect code clones, Wang, et al. [29] designed a graph representation of programs called a flow-augmented AST and applied two distinct kinds of graph neural networks to it. First, they parsed the pair of code segments into an AST. The graph was represented by adding data and control flow edges to the AST. Vectors were created for both pairs of code segments from the representation. It was then converted into a graph-level vector for each segment, the code representation for detecting code clones. Cosine similarity was applied to detect code clones. They were able to detect all forms of code clones. Google Code Jam and BigCloneBench were utilized as datasets in this study.

Sheneamer et al. [15] developed a CCD approach (Pairwise Feature Fusion CCD) to identify all four types of clones using ML algorithms. First, they trimmed and normalized the source code. Afterwards, the normalized source code method pairs were parsed into AST and PDG, and features were extracted from those representations—frequency of programming constructs. Feature vectors created using both AST and PDG were fused into a single feature vector using three distinct ways—linear combination, multiplicative combination, and distance combination. These significant steps were taken as source code transformation/normalization. Ultimately, the source code was represented as a vector before being applied to the ML algorithms. Their approach was able to classify

all types of clones. Clones were detected pairwise. IJaDataset 2.0, eclipse-ant, eclipse-jdtcore, netbeansjavadoc, j2sdk14.0-javax-swing, EIRC, and Suple were the datasets they utilized. They extracted a total of 100 features, including line count, if statements count, and assignment count. More information on extracted features can be found in their supplementary material. They used fifteen ML algorithms, including Convolutional Neural Network (CNN).

Various features from the source code in the past were considered to detect code clones based on the related works listed above. However, none of the previous studies utilized the 20 features that were used in this study in detecting code clones. In this study, a total of 48 features were extracted using AST. Many studies utilized ML and DL in detecting code clones. However, none of the researchers used the LeONet to detect code clones in this study. Therefore, combining the features retrieved from AST, coupled with this proposed hybrid DL approach, can provide a solid foundation for detecting code clones.

3. Methodology

3.1. Research Design

The entire methodological framework and all the steps in this study are depicted in Figure 2. Each step has been thoroughly explained below.

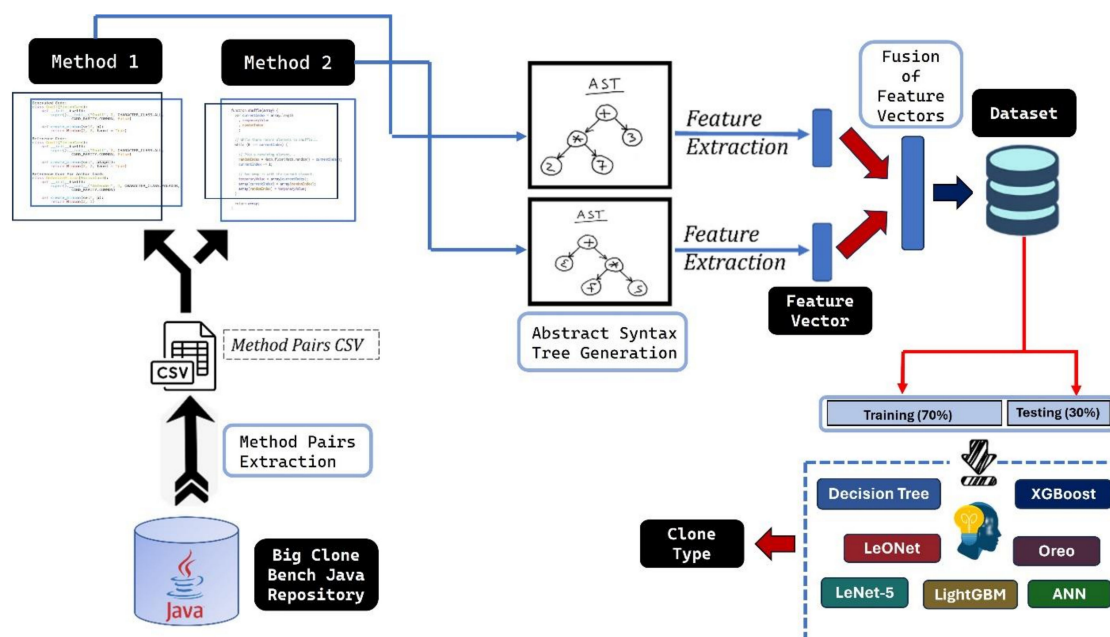


Figure 2. Research design.

3.1.1. BigCloneBench Java Repository

This study uses BigCloneBench [30–32], a Java repository containing clones known to be true and false positives. It is a well-known repository in CCD studies [20,33–35].

It contains all four types of code clones—T1, T2, T3, and T4. Since there is no firm agreement on the minimum syntactical similarity for T3 clones, it is difficult to distinguish between T3 and T4 clone pairs. As a result, researchers established four categories using syntactical similarities. They are Very-Strongly Type-3 (VST3), Strongly Type-3 (ST3), Moderately Type-3 (MT3), and Weakly Type-3/Type-4 (WT3/4). VST3 spans from 90% (inclusive) to 100% (exclusive), ST3 ranges from 70 to 90%, MT3 ranges from 50 to 70%, and WT3/4 ranges from 0 to 50% [31,32]. Therefore, the repository distinguishes clone types as T1, T2, VST3, ST3, MT3, and WT3/4.

False clone pairs are also available in the BigCloneBench. False clone pairs are syntactically dissimilar code fragments that do not compute the same functionality (functionally dissimilar code fragments). The syntactical similarity between false clone pairs is coincidental [30]. For example, the original method computes the square of the given value, whereas the false clone method calculates the sum of the two values. Here, both methods' functionalities are different—functionally dissimilar. Therefore, they can be considered false clone pairs.

3.1.2. Method Pairs Extraction

The Comma-Separated Value (CSV) file was extracted from the BigCloneBench repository. A total of 4190 method pairs were extracted for each clone type. The criteria for selecting the clone pairs were based on the first 4190 unique pairs from the repository. Then, for all pairs of methods in each method pair of CSV files listed above, ASTs were generated.

3.1.3. AST Generation

AST is a tree description of a computer program's source code. It is made up of nodes. Each node represents a programming construct occurring in the source code. The entire node set describes the structure of the source code [36].

Figure 3 describes the AST view of a Java method. The “MethodDeclaration” node encapsulates all programming constructs of the “getSquare” method. When considering the “name” node, which is directly connected with the “MethodDeclaration” node, it represents a name property whose type is “SimpleName”, and it is associated with the given method's name. Further deep down to the next level of node “identifier = ‘getSquare’”, it represents the identifier property whose simple value is “getSquare”, which is the name of the given method. Similarly, when considering the “type” node, which is directly connected with the “MethodDeclaration” node, it represents a type property whose datatype is “PrimitiveType”, and it is associated with the given method's return type. Moving to the next level of the node “type = ‘INT’”, it represents the type property whose simple value is “INT”, which is the actual return type of the given method “getSquare”. ASTs were generated for paired methods using Eclipse JDT libraries [37] and JavaParser [38]. After generating ASTs, features were extracted from them. In Java, the * symbol is used to represent multiplication.

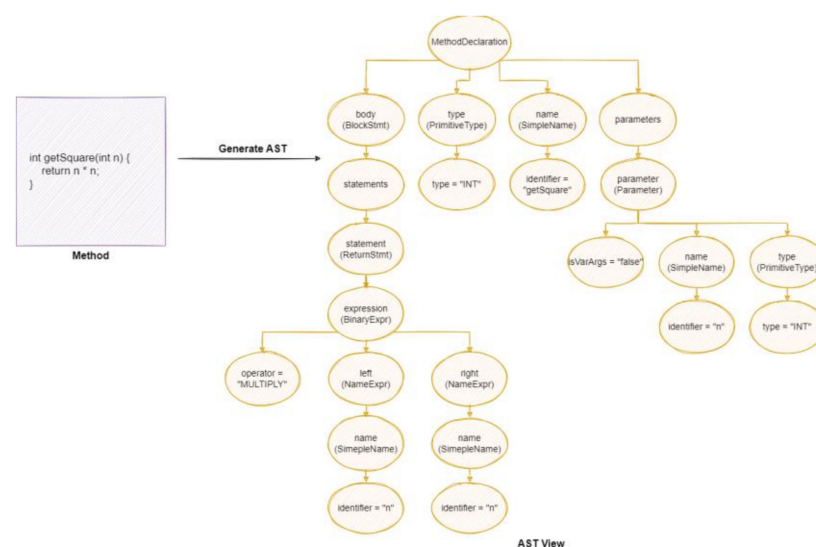


Figure 3. AST view of the Java model.

3.1.4. Feature Extraction

A total of 48 features were extracted using AST. These 48 features can be divided into two sets: (i) 28 prevailing features used in the previous studies [15] and (ii) 20 features that were not considered in the previous studies.

When considering 28 prevailing features, the author extracted a total of 28 features listed under the traditional (14 features) and AST (14 features) categories specified in the Supplementary Material [15]. Table 1 describes all the features extracted in this study. The first 28 features are prevailing, whereas the next 20 are new. Features are represented in whole numbers, that is, numbers from 0, 1, 2, 3, 4, and so on.

Table 1. Name and representation of the features.

No	Feature	Representation	No	Feature	Representation
1	Lines count	Whole number	25	Primitive types count	Whole number
2	Assignments count	Whole number	26	Simple names count	Whole number
3	Selection statements count	Whole number	27	Simple types count	Whole number
4	Iteration statements count	Whole number	28	Wildcard types count	Whole number
5	Synchronized statements count	Whole number	29	Postfix expressions count	Whole number
6	Return statements count	Whole number	30	Variable declaration fragments Count	Whole number
7	Switch case statements count	Whole number	31	Reference types count	Whole number
8	Try statements count	Whole number	32	Void types count	Whole number
9	Single variable declarations count	Whole number	33	Binary expressions count	Whole number
10	Variable declarations count	Whole number	34	Double literal expressions count	Whole number
11	Variable declaration statements count	Whole number	35	Integer literal expressions count	Whole number
12	Expression statements count	Whole number	36	Long literal expressions count	Whole number
13	Type declaration statements count	Whole number	37	Literal string value expressions count	Whole number
14	Type parameters count	Whole number	38	Unary expressions count	Whole number
15	Class instance creations count	Whole number	39	Type bounds count	Whole number
16	Array creations count	Whole number	40	Boxed types count	Whole number
17	Cast expressions count	Whole number	41	Array creation levels count	Whole number
18	Constructor invocations count	Whole number	42	Poly expressions count	Whole number
19	Field declarations count	Whole number	43	Standalone expressions count	Whole number
20	Super method invocations count	Whole number	44	Elided type arguments count	Whole number
21	Infix expressions count	Whole number	45	Qualified expression names count	Whole number
22	Method invocations count	Whole number	46	Simple expression names count	Whole number
23	Method refs count	Whole number	47	Primary expressions count	Whole number
24	Parenthesized expressions count	Whole number	48	Literal expressions count	Whole number

Table 2 describes the new features and their code examples. The examples provided for each feature are illustrative but not exhaustive. They include, but are not limited to, the given examples.

Table 2. New feature with examples.

No	Feature	Example
1	Postfix expressions count	<code>x++</code> ; (<code>x++</code> is a postfix expression)
2	Variable declaration fragments count	<code>int x = 10, y = 20, z = 30</code> ; (<code>x, y, and z</code> are variable declaration fragments)
3	Reference types count	<code>Map<String, Integer> map = new HashMap<>()</code> ; (<code>Map<String, Integer></code> is a reference type)
4	Void types count	<code>void method()</code> (The void type indicates that the method does not return a value)
5	Binary expressions count	<code>boolean result = (a > 0 && b < 10)</code> ; (<code>a > 0 && b < 10</code> is a binary expression)
6	Double literal expressions count	<code>double pi = 3.14</code> ; (<code>3.14</code> is a double literal expression)
7	Integer literal expressions count	<code>int count = 42</code> ; (<code>42</code> is an integer literal expression)
8	Long literal expressions count	<code>long value = 123456789L</code> ; (<code>123456789L</code> is a long literal expression)
9	Literal string value expressions count	<code>String response = "Success": + statusCode</code> ; (<code>"Success":</code> is a literal string value expression)
10	Unary expressions count	<code>boolean result = !flag</code> ; (<code>!flag</code> is a unary expression)
11	Type bounds count	<code>class Container<T extends Comparable<T>></code> (the bound <code>Comparable<T></code> specifies that <code>T</code> must implement <code>Comparable<T></code>)
12	Boxed types count	<code>Integer x = 5</code> ; (the type <code>Integer</code> is the boxed type)
13	Array creation levels count	<code>int[][] grid = new int [3][4]</code> ; (2D array creation with 2 levels)
14	Poly expressions count	<code>int result = x + (y > 10? 5:3)</code> ; (The expression <code>y > 10? 5:3</code> is a poly expression within the addition <code>x + (y > 10? 5:3)</code>)
15	Standalone expressions count	<code>if (a > b) System.out.println("A is greater")</code> ; (the entire if statement is a standalone expression)
16	Elided type arguments count	<code>exampleMethod();, obj.exampleMethod();, obj<>.exampleMethod();</code>
17	Qualified expression names count	<code>java.util.Date today = new java.util.Date()</code> ; (<code>java.util.Date</code> is a qualified expression name)
18	Simple expression names count	<code>int result = a + (b * (c - d))</code> ; (<code>a, b, c, and d</code> are simple expression names)
19	Primary expressions count	<code>int max = Math.max(a, b)</code> ; (<code>Math.max</code> is a primary expression)
20	Literal expressions count	<code>boolean flag = false</code> ; (<code>false</code> is a literal expression—Boolean literal)

For instance, say method one and method two are considered paired methods. Once the features from method one is extracted, put them into a vector. Similarly, when extracting the features from method two, put them into a vector. Consequently, extracted features of paired methods M_i and M_j can be represented as feature vectors: $M_i = \langle f_{i1}, f_{i2}, \dots, f_{ik} \rangle$, and $M_j = \langle f_{j1}, f_{j2}, \dots, f_{jk} \rangle$. Then, the two feature vectors representing a paired method were combined into a single fused vector along with its associated clone type.

3.1.5. Fusion of Feature Vectors

A single vector was created by combining a pair of feature vectors, followed by the relevant class label. The class label was the clone type (output). The fused feature vector can be expressed as $features(\langle M_i, M_j \rangle)$, where M_i and M_j , are the given paired methods and their clone type $C1$. Two vectors were combined using the linear and distance combination strategies [15]. The Distance combination approach is selected for non-Siamese architectures, whereas the linear combination approach is selected for Siamese architectures.

The distance combination approach calculates the absolute difference between two associated values of a feature. Ultimately, fused feature vector using distance combination strategy for a pair of methods with their clone type can be expressed as $features(< M_i, M_j >) = < |f_{i1} - f_{j1}|, \dots, |f_{ik} - f_{jk}|, C1 >$, where $C1$ represents the clone type. The linear combination strategy is carried out by concatenating the two associated feature vectors. A fused feature vector using a linear combination strategy for a pair of methods with its clone type can be expressed as $features(< M_i, M_j >) = < f_{i1}, \dots, f_{ik}, f_{j1}, \dots, f_{jk}, C1 >$, where $C1$ represents the clone type.

3.1.6. Dataset

Fused feature vectors were produced into two CSV files. For example, fused feature vectors of T1 using the linear combination strategy were produced into a CSV file, and fused feature vectors of T1 using the distance combination strategy were produced into a CSV file. Figure 4 describes the transformation that happens regardless of the combination strategy.

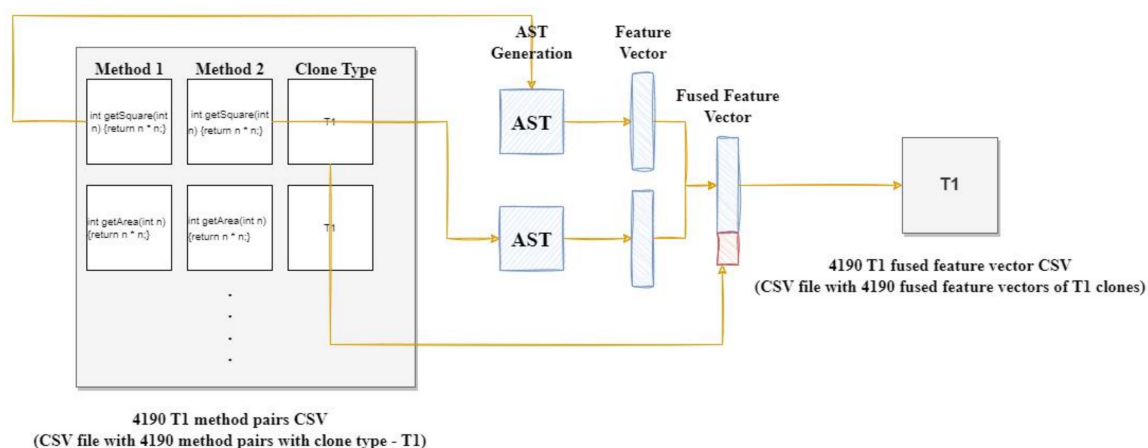


Figure 4. Transformation from 4190 method pairs CSV to 4190 fused feature vector CSV.

Figure 4 describes the transformation from 4190 T1 method pairs CSV to 4190 T1 fused feature vector CSV. In the 4190 T1 method pairs CSV file, the author parsed method pairs to AST, extracted the features, and fused the feature vectors with their associated clone type. These fused feature vectors were produced into a 4190 T1 combined feature vector CSV file.

Similarly, the remaining six clone types (T2, VST3, ST3, MT3, WT3/4, False) of the 4190 method pairs CSV were processed to the 4190 fused feature vector CSV. Ultimately, seven 4190 fused feature vector CSVs were used for each combination strategy. Those seven fused feature vector CSVs were combined into a single CSV file for each combination strategy to construct the dataset. These two CSV files were the dataset—the linear dataset and the distance dataset. The distance dataset consists of 48 features and a clone type. It contains 29,330 rows ($7 * 4190 = 29,330$) of fused feature vectors. The linear dataset consists of 96 features and a clone type. It contains 29,330 rows ($7 * 4190 = 29,330$) of fused feature vectors.

The models developed in this study were not only trained to detect clone pairs but also false clone pairs. For instance, in Figure 5, the original method computes the square of the given value, whereas the false clone method calculates the sum of the two values. Such pairs are labelled as False.

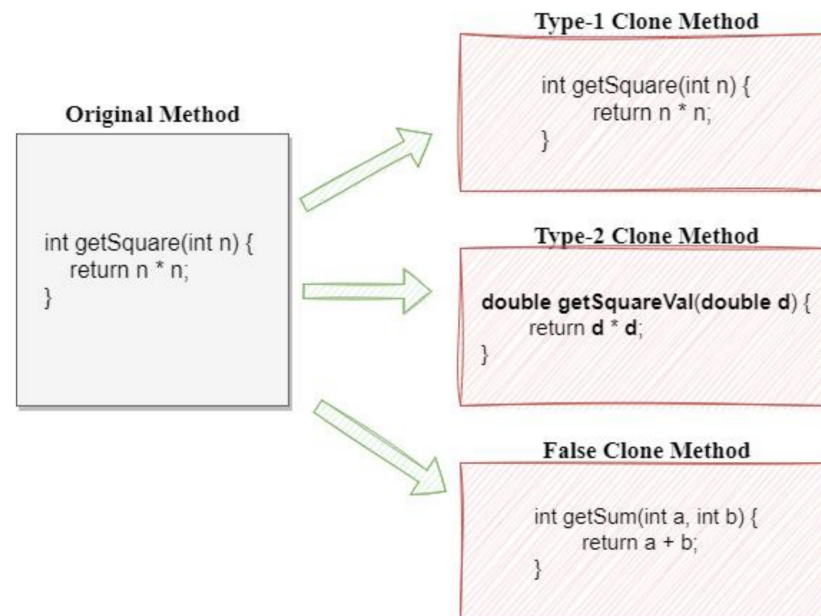


Figure 5. Example of Type-1 and Type-2 clone methods.

Features of this study are the frequency of different characteristics taken from methods, and there is no difference between T1 and T2 clones apart from literal values, identifier names, and types [2]. Literals, identifiers, and types cannot be differentiated using the features utilized in this study.

In Figure 5, the original method “getSquare” calculates and returns an integer type square value for the given integer number “n”. The T1 clone method is the same as the original method. T2 clone method “getSquareVal” calculates and returns double type square value for the given double value “d”. The T1 clone method and T2 clone method can be differentiated from their original method based on identifiers and types. Apart from these differences, the frequency of different characteristics taken from methods cannot distinguish them.

To illustrate more, in Table 3, features were extracted for the above original, T1, and T2 methods individually, and useful feature details were listed.

Table 3. Feature extraction values.

Feature	Original Method	T1 Clone Method	T2 Clone Method
Lines count	3	3	3
Return statements count	1	1	1
Single variable declarations count	1	1	1
Variable declarations count	1	1	1
Infix expressions count	1	1	1
Primitive types count	2	2	2
Simple names count	5	5	5
Binary expressions count	1	1	1
Standalone expressions count	3	3	3
Elided type arguments count	3	3	3
Simple expression names count	2	2	2

Table 3 shows that the features and frequency of different characteristics are the same. Thus, distinguishing between the T1 and T2 clone methods from the original methods is difficult. Therefore, it is impossible to differentiate between two feature vectors from a pair of T1 or T2 methods when utilizing a distance or linear combination strategy.

The author performed a multiclass classification task to distinguish all clone types, including T2 clones. The multiclass classifier struggled to distinguish between T1 and T2 clones. The model completely failed to correctly predict any T1 instances, resulting in zero precision, recall, and F1 score for that class. Meanwhile, the T2 class achieved perfect recall 1.00 but moderate precision 0.46 and an F1 score of 0.63, indicating that many T1 samples were misclassified as T2 clones. This pattern highlights a fundamental confusion between T1 and T2, an aspect not effectively captured by our current feature set. Therefore, the model tends to assign T1 instances to the T2 class, which acts as a catch-all category. These results underscore the need for incorporating identifier-sensitive or semantic features to improve fine-grained clone classification.

As a result, T1 clone pairs were solely considered. However, the rest of the clone types can be discovered through this study. This study's shortcoming is that T2 clones were not identified alongside the other clones. Therefore, 4190 T2 fused feature vector data was removed from the dataset.

Accordingly, this study focuses on detecting six clone types: T1, VST3, ST3, MT3, WT3/4, and False. As a result, the linear and distance dataset consists of 25,140 rows ($6 * 4190 = 25,140$) of fused feature vectors. The dataset is balanced, with each type having 4190 instances.

While 70% of the dataset (17,598 sets of data) was utilized for training a multiclass classification model, the remaining 30% (7542 sets of data) was preserved for testing purposes. Due to the randomness, at an instance, the test data contained 1273, 1267, 1262, 1261, 1243, and 1236 clones for types T1, VST3, ST3, MT3, WT3/4, and False, respectively. Test sets were inputted to the trained model, and its performance was evaluated using evaluation metrics. The model evaluation metrics utilized were recall, precision, and F1 score.

3.1.7. Multiclass Classifier Model

Various multiclass classifier models were implemented to classify code clone types effectively in this study. They are Artificial Neural Network (ANN), LeNet-5 [18], Oreo's Siamese architecture [19], Light Gradient-Boosting Machine (LightGBM) [39], XGBoost [40], Decision Tree, and LeONet.

3.1.8. Clone Type

Eventually, the multiclass classification models implemented would output the clone type. The output can be either T1, VST3, ST3, MT3, WT3/4, or False. T1 represents T1 clone pairs, VST3 represents VST3 clone pairs, ST3 represents ST3 clone pairs, MT3 represents MT3 clone pairs, WT3/4 represents WT3/4 clone pairs, and False represents false clone pairs.

3.2. Proposed Hybrid DL Approach

3.2.1. LetNet-5

LeCun et al. [18] proposed the LeNet-5 architecture. It is a CNN that has eight layers, including the input layer. Figure 6 depicts the LeNet-5 architecture used in this study.

The distance dataset served as the model's input, a 49-dimensional vector (48 features and a clone type). Then it moved into three convolutional layers (C1, C2, C3), two average pooling layers (P1, P2), a fully connected layer, and an output layer. In Figure 6, "f" represents filters, the filter size is represented by "fs", "s" represents stride, and the size of the feature map is represented by "fm". The model was trained for 500 epochs with a batch size of 32. Table 4 describes the parameter settings configured for LeNet-5 in this study.

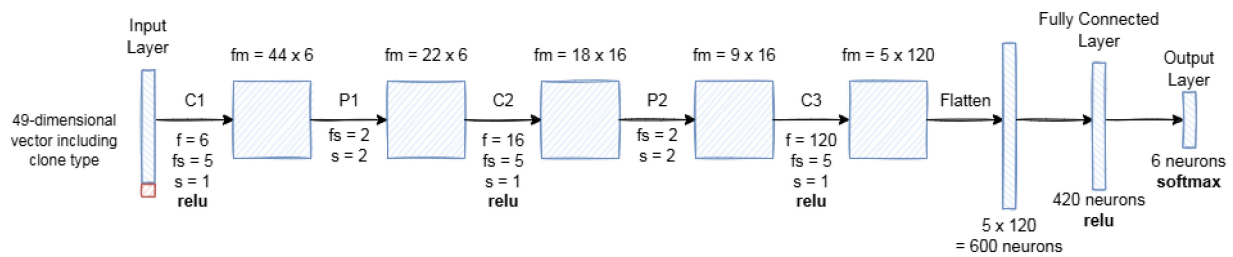


Figure 6. LetNet-5 architecture model.

Table 4. Parameter settings for LetNet-5.

Layer	Filters	Kernel Size/Pool Size	Stride	Size of Feature Map	Activation Function
Input	-	-	-	48	-
Convolutional 1	6	5	1	44×6	relu
Pooling 1	-	2	2	22×6	-
Convolutional 2	16	5	1	18×16	relu
Pooling 2	-	2	2	9×16	-
Convolutional 3	120	5	1	5×120	relu
Flatten	-	-	-	$5 \times 120 = 600$	-
Fully Connected 1	-	-	-	420	relu
Output	-	-	-	6	softmax

3.2.2. Oreo

Figure 7 depicts Oreo's Siamese architecture model used in this study, which was proposed by Saini et al. [19].

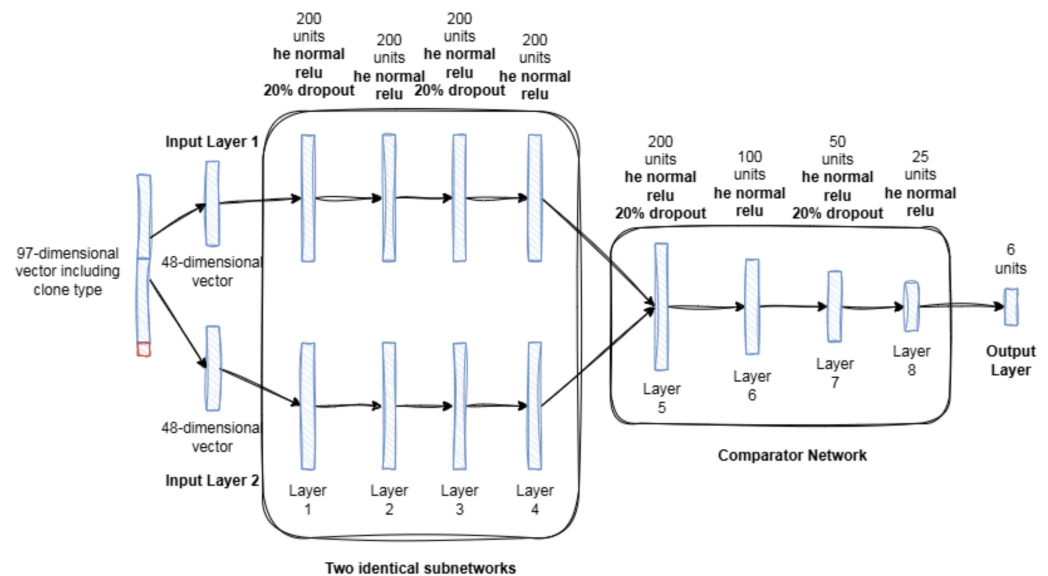


Figure 7. Oreo Siamese architecture model.

The input to this model came from the linear dataset, which consisted of a 97-dimensional vector. This vector included 48 features from each method, along with the clone type. Each subnetwork received 48 features from one method, while its identical subnetwork received 48 features from the other method. Two identical subnetworks—sister networks—mirror each other. The subnetwork had four fully connected hidden layers of 200 units. Units are the same as neurons. The comparator network then received the combined outputs of the two subnetworks. The comparator network had four fully connected

layers of units 200, 100, 50, and 25. The output layer received the output of the comparator network. Six units comprise the output layer since there are six clone types. A 20% dropout was applied to every other layer in the network [19]. The model was trained for 500 epochs with a batch size of 32. Table 5 details the parameter settings of the subnetwork, while Table 6 outlines the parameter settings of the comparator network used in this study.

Table 5. Parameter settings for the subnetwork.

Layer	Units	Kernel Initializer	Activation Function
Dense	200	he normal	relu
Dropout (20%)	-	-	-
Dense	200	he normal	relu
Dense	200	he normal	relu
Dropout (20%)	-	-	-
Dense	200	he normal	relu

Table 6. Parameter settings for the comparator network.

Layer	Units	Kernel Initializer	Activation Function
Dense	200	he normal	relu
Dropout (20%)	-	-	-
Dense	100	he normal	relu
Dense	50	he normal	relu
Dropout (20%)	-	-	-
Dense	25	he normal	relu

3.2.3. LeONet

The author employed ML algorithms with 48 features in detecting code clones. They produced encouraging results in CCD. Due to nature of comparing entities with the Siamese architecture while maintaining the balance between performance and efficiency, and the fact that Saini et al. [19] implemented Siamese architecture in OreO with motivating results reported on their paper in detecting code clones, the author started to investigate OreO's Siamese architecture [19,41–43]. OreO's Siamese model uses four fully connected layers as their subnetwork. Subnetworks output their learning to the comparator network, which is then compared, and the clone type is eventually classified [19]. Siamese subnetworks are used to extract useful feature representations [44,45]. The more effective the feature representation, the more advantageous it will be for the comparator network in comparing and classifying code clones.

Feature representations of CNNs are highly versatile and effective [46,47]. Their feature representation and learning would be better than the four fully connected layers that were used in OreO. Using them as subnetworks could ideally improve the performance in CCD. Among the CNNs, the author considered LeNet-5 due to its lightness in architecture, capacity of memory, and computational complexity [48]. As a result, the proposed hybrid model LeONet is made up of OreO's Siamese architecture and LeNet-5 architecture.

Siamese architecture offers flexibility in configuration of networks according to the task, including the utilization of various neural network types, such as CNNs. Furthermore, they require less training data and are less sensitive to overfitting compared to other architectures [41]. The proven effectiveness of the Siamese architecture in various fields like face recognition [49] and signature verification [50], object tracking [51], source-code vulnerability detection [52], including CCD [53,54], makes it more advantageous to use compared to recent DL models.

The reason for LeNet-5's popularity lies in its simplicity and straightforward architecture [55]. LeNet-5 architecture shines in diverse fields like traffic sign recognition [56], tumor diagnosis [57,58], and handwritten digit classification [59], which establishes its promising effectiveness for CCD. Along with these, its low computational complexity [48] makes it more beneficial to use in a hybrid architecture for CCD compared to recent DL models.

While designing LeONet, the author initially explored the feasibility of integrating other CNN architectures. However, the input consists of one-dimensional vectors, derived from AST-based structural features of source code. When these models were adapted to this study's low-dimensional dataset, the deeper CNNs, with their multiple stacked convolutional and pooling layers, failed to train effectively. As a result, LeNet-5 was chosen for its simplicity and adaptability in this context, making it a practical and well-suited architectural choice for the twin network.

Figure 8 describes the LeONet architecture. In this architecture, the subnetwork is LeNet-5, whereas the comparator network is the same as Saini et al. [19] mentioned. LeONet uses the same parameters described in Tables 4 and 6.

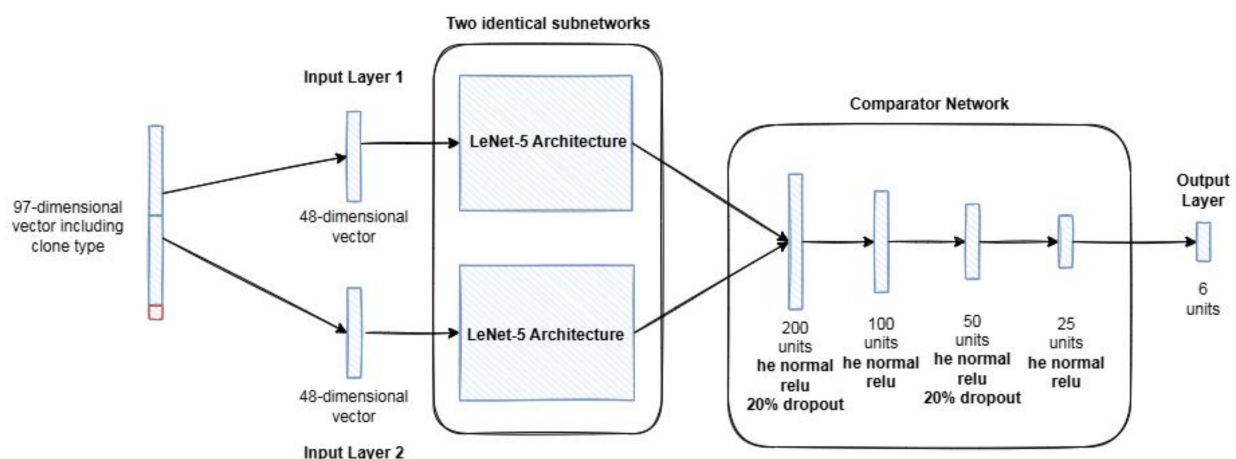


Figure 8. LeONet architecture model.

The linear dataset was fed into this model. Forty-eight features of a method were provided to the LeNet-5 subnetwork, and its paired method's forty-eight features were provided to the identical subnetwork. The same transformations were applied to both inputs by these two identical subnetworks. These subnetworks' output was combined and supplied to the comparator network. Units 200, 100, 50, and 25 were spread among four fully connected layers of the comparator network. The output of the comparator network was then sent to the output layer. The output layer had six units since there are six clone types to be considered. The model was trained for 500 epochs with a batch size of 32.

As mentioned in Table 7, the Oreo model [19] and LeONet used the linear dataset. Saini et al. [19] proposed the Oreo model, which utilizes a Siamese architecture. They created a 48-dimensional vector consisting of 24 metrics from each method in a pair (linear dataset), totaling 48 features. Each subnetwork received 24 features from one method, while its identical subnetwork received 24 features from the other method. Both identical subnetworks of the model applied the same transformation to these input vectors.

LeONet is similar to the Oreo Siamese architecture model [19]. The distance dataset cannot be used for the Oreo model and LeONet since they need to have the same features for the two identical subnetworks. Because of that, in this study, the Oreo model and LeONet were trained on a linear dataset.

Table 7. Models and dataset used.

No	Models	Dataset Used	No	Models	Dataset Used
1	ANN	Distance dataset	5	XGBoost	Distance dataset
2	LeNet-5	Distance dataset	6	Decision Tree	Distance dataset
3	Oreo	Linear dataset	7	LeONet	Linear dataset
4	LightGBM	Distance dataset			

The models are designed to learn from and produce results based on a set of 48 programming constructs. Although this study employs different datasets for various models, all models ultimately process and classify features derived from the same set of 48 programming constructs.

LeNet-5 receives 48 features as a single input from the distance dataset and performs classification based on these features. Since LeNet-5 is not a Siamese architecture, inputting the same feature twice (e.g., line count features from both method 1 and method 2 in the linear dataset) into LeNet-5 could lead to ambiguity. LeNet-5 is designed to learn from distinct features and classify inputs based on their unique representations. Therefore, LeNet-5 cannot be used with linear datasets. In contrast, Siamese architectures like LeONet and Oreo's models are specifically designed for comparing features from pairs of inputs, such as from a linear dataset. Consequently, the results from each model are compared based on their respective ability to detect code clones and effectiveness in handling the provided features.

Even though the proposed approach utilizes a linear dataset suited for Siamese architectures, the distance dataset was developed to serve as a benchmark for comparing the performance of different models. While the proposed approach uses a linear dataset, the distance dataset allows for us to assess how well-known models, like LeNet-5, LightGBM, and XGBoost, perform in detecting code clones.

4. Results and Discussion

4.1. Machine Configuration

All experiments were carried out on a machine with the following specifications: 8 GB of RAM, an Intel Core i5-8250U CPU, an NVIDIA GeForce MX130 GPU, a 1000 GB hard disk drive, running Python 3.10.13 and TensorFlow 2.10.0, and operating on Windows 11.

4.2. Model Evaluation Metrics

Three metrics are used to evaluate the performance of the models in the experiments: *Recall*, *precision*, and *F1 score*. *Recall* indicates the model's ability to cover the clone samples [60].

$$Recall = \frac{True\ Positive}{True\ Positive + False\ Negative} \quad (3)$$

Equation (3) describes the recall equation used in this study [60]. *True positive* is a number of correctly classified samples as positive samples, and *false negative* is the number of positive samples misclassified as negative samples.

Precision measures the accuracy of the model in classifying positive samples. It is calculated using Equation (4). *False positive* is the number of incorrectly classified samples as positive samples [60].

$$Precision = \frac{True\ Positive}{True\ Positive + False\ Positive} \quad (4)$$

F1 Score combines *Precision* and *Recall*, enabling a better evaluation of the model's detection performance [60].

$$F1\ Score = \frac{2 * Precision * Recall}{Precision + Recall} \quad (5)$$

F1 score is measured using Equation (5) [60].

4.3. Performance Comparison of Several Algorithms Used in This Study to Detect Code Clones

Figure 9 illustrates the recall, precision, and F1 score performances of all seven models utilized in this study. LeONet outperformed all the models that were developed in this study.

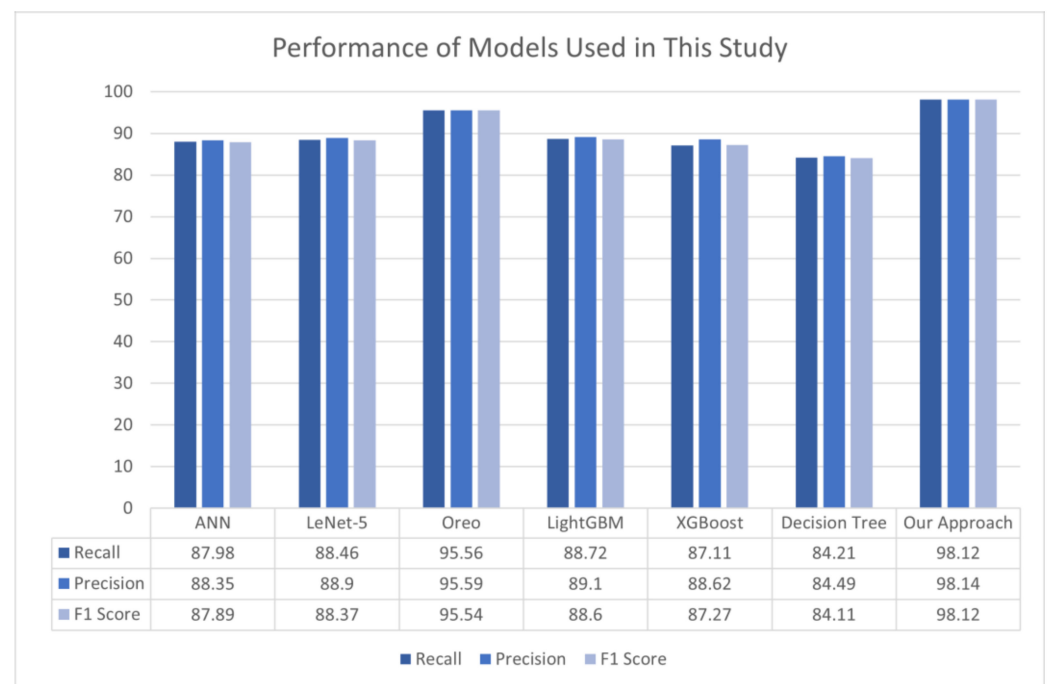


Figure 9. Performance of models used in this study.

In Figure 9, LeONet's performance gain over Oreo can be attributed to the integration of LeNet-5, which acts as a preprocessing subnetwork that transforms the features. This enables LeONet to capture richer structural representations than Oreo alone.

Figure 10 illustrates the recall performance comparison of various algorithms employed in this study by clone type. Traditional models such as ANN, LeNet-5, XGBoost, and Decision Tree achieve perfect recall for Type 1 clones, which are syntactically identical and easier to detect, followed closely by LeONet with 97.82% recall. This drop can be attributed to LeONet's emphasis on learning generalizable structural patterns rather than focusing solely on exact syntactic matches. As a deep hybrid model operating on AST-derived features, LeONet may abstract away low-level syntactic details, which can occasionally affect its ability to perfectly identify exact matches. However, ANN, LeNet-5, XGBoost, and Decision Tree performance declines with more structurally varied clones. LeONet, in contrast, maintains strong and consistent recall across all clone types, achieving top scores for VST3 (97.15%), ST3 (95.26%), and competitive recall for WT3/4 (99.76%) and false clones (100%). This indicates LeONet's ability to generalize beyond exact syntax, making it more robust for complex, near-miss clones. Although LightGBM achieves perfect recall for MT3, and Oreo for WT3/4, their results are more isolated, suggesting less consistency across clone categories.

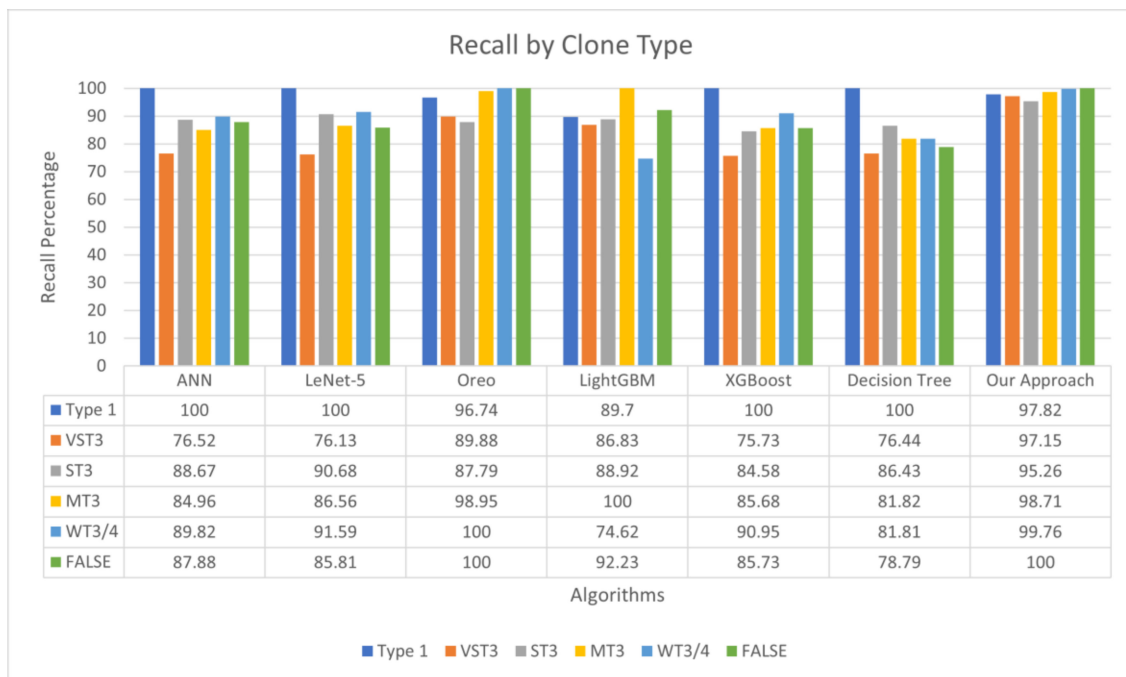


Figure 10. Recall by clone type.

Figure 11 demonstrates the precision performance comparison of the algorithms utilized by clone type. The precision results reveal that LeONet maintains consistently high performance across all clone types, achieving near-perfect precision for Type 1 (99.84%), WT3/4 (100%), and false clones (99.76%). This indicates a strong ability to avoid false positives, particularly in structurally complex and unrelated code. While Oreo slightly edges out LeONet in a few categories, LeONet demonstrates greater balance across VST3 (94.54%), ST3 (97.13%), and MT3 (97.54%) clones, where other models show more variability. These results highlight LeONet's strength in reliably distinguishing true clones from non-clones across a wide range of syntactic and semantic similarity levels.

Figure 12 describes the F1 score performance comparison of the algorithms utilized by clone type.

The F1 Scores across clone types show that LeONet consistently outperforms baseline models, demonstrating strong generalizability across both simple and complex clones. Its high performance on VST3, ST3, and MT3 indicates the model's ability to capture structural similarities beyond surface-level syntax. This advantage stems from its hybrid twin-network design, which combines structural AST features with deep representation learning. In contrast, traditional models perform well on simpler clones but struggle with structurally transformed ones. Overall, LeONet strikes an effective balance between precision and recall, making it a robust solution for diverse clone detection scenarios.

The 30% test set was used for evaluating performance across different clone types, and the author did not utilize separate validation sets for each clone type. The recall, precision, and F1 score for each clone type were computed using scikit-learn's [61] `recall_score`, `precision_score`, and `f1_score` functions with `average = None`. This setting calculates the metrics for each class individually, rather than aggregating them into a single value.

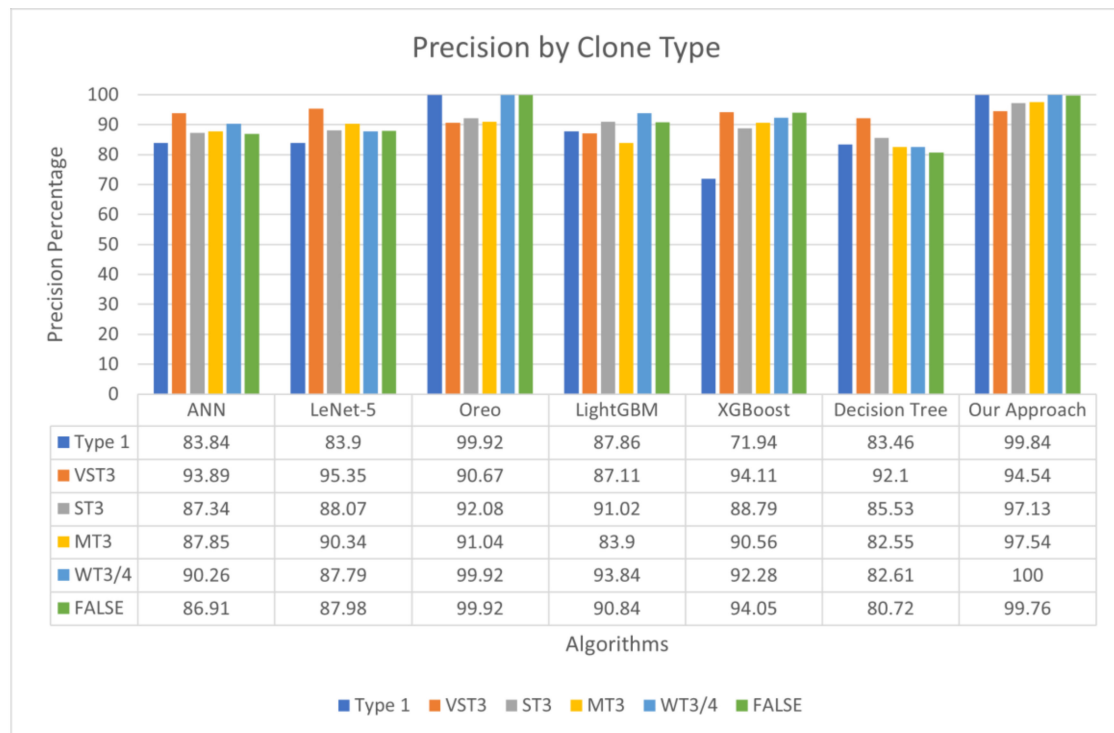


Figure 11. Precision by clone type.

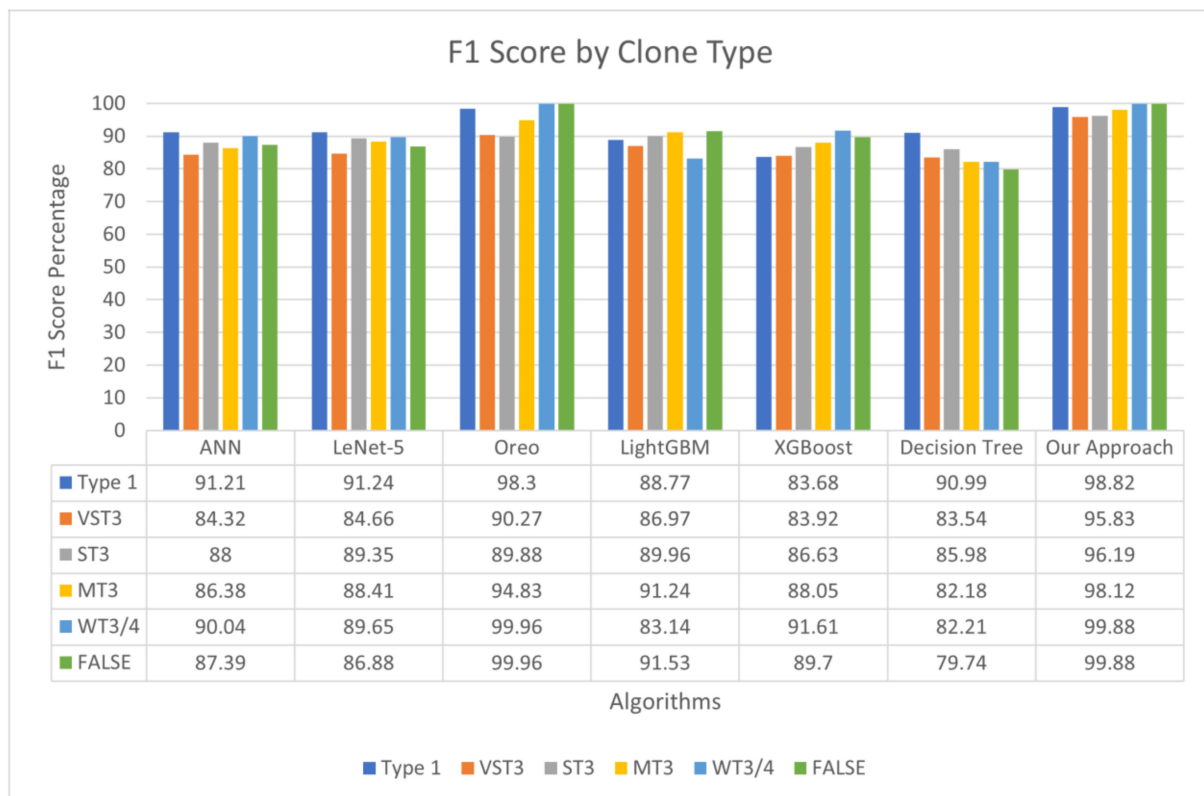


Figure 12. F1 score by clone type.

4.4. Performance Comparison of Several CCD Approaches Considered in Literature Review

The overall clone detection performance of LeONet compared with nine state-of-the-art approaches was evaluated. The purpose of this comparison is to demonstrate the significance of this study.

The considered state-of-the-art approaches were SourcererCC proposed by Sajjani, et al. [22], RtvNN proposed by White et al. [23], Deckard by Jiang et al. [24], DeepSim [25], SCDetector [26], ASTNN proposed by Zhang et al. [27], TreeCen [28], FA-AST+GMN developed by Wang et al. [29], and Semantic CCD by Sheneamer et al. [15].

BigCloneBench results of SourcererCC, RtvNN, and Deckard were obtained from [28]. Other results were obtained from their corresponding research papers.

Figure 13 illustrates a performance comparison of several CCD approaches using recall, precision, and F1 score evaluation metrics. TreeCen showed top recall performance (0.99), followed by LeONet (0.98) and DeepSim (0.98). Top Precision performance was demonstrated by ASTNN (1.0), followed by TreeCen (0.99), LeONet (0.98), and SCDetector (0.98). When considering the F1 score, TreeCen expressed good performance (0.99), followed by LeONet (0.98), SCDetector (0.98), and DeepSim (0.98).

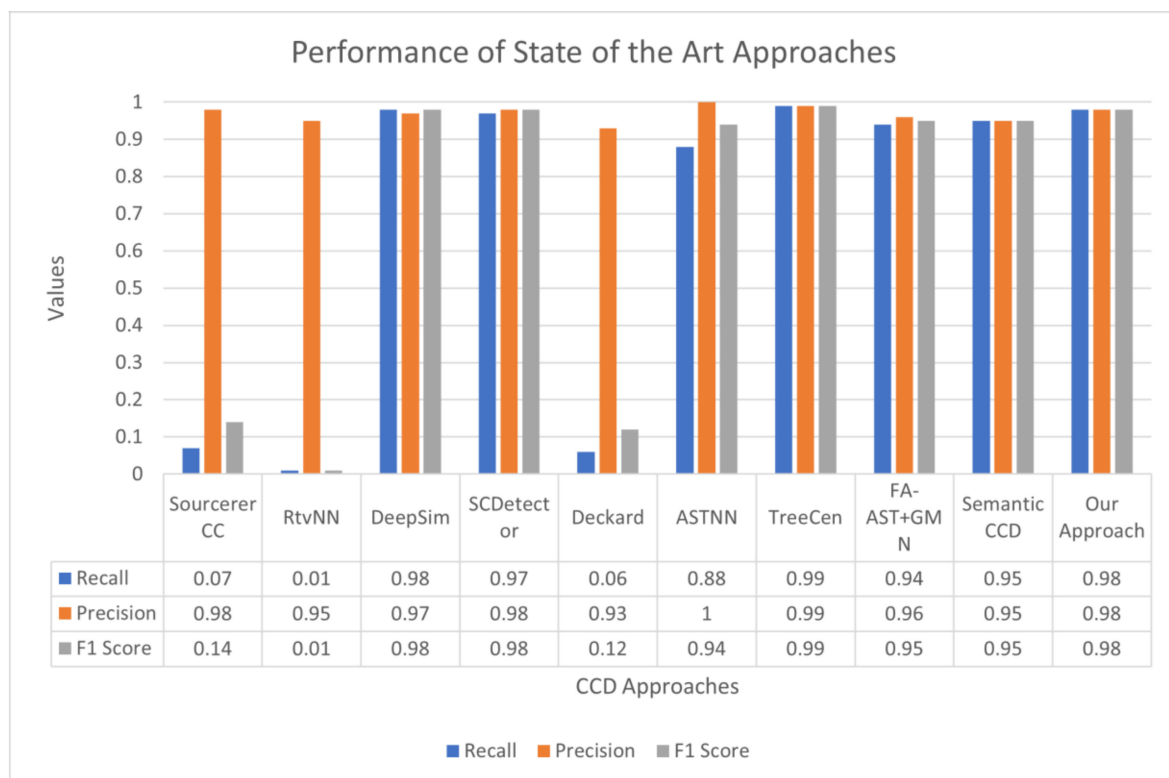


Figure 13. Performance of state of the art approaches.

4.5. Results Obtained During Evaluation of Features in Classifying Code Clones

Features were evaluated in two ways: (i) F1 score performance of different algorithms with an increasing number of features to describe the importance of the 48 features used in this study and (ii) F1 score performance comparison of different algorithms using the prevailing 28 features and the 20 features that were not considered in previous studies to describe the importance of the 20 features that were not considered in previous studies in getting good model performance and that this is not accidental.

Figure 14 demonstrates the F1 score performance of different algorithms with increasing features. The feature count started at 8 and increased by increments of 8 until it reached 48. As can be seen, all four models' F1 Scores increased as the number of features increased. This shows the significance of the features that were used in this study to detect code clones.

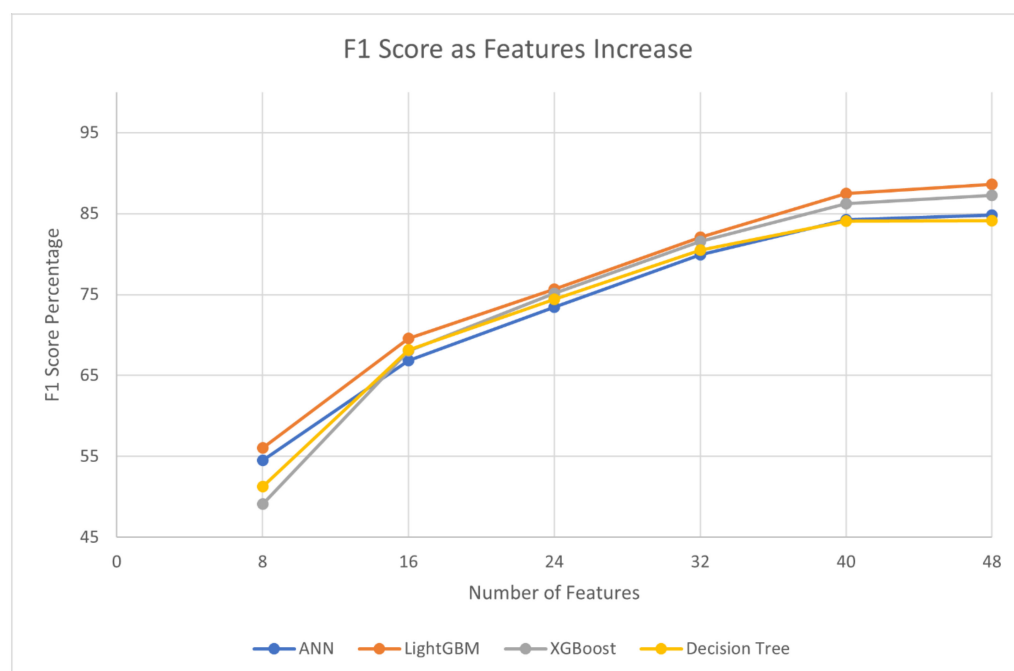


Figure 14. F1 score as feature increases.

The impact of the additional 20 new features on CCD performance can be measured by analyzing how various algorithms perform on the existing feature set and the new feature set. Feature sets were evaluated using well-known models. Figure 15 illustrates the F1 score performance comparison of different algorithms between the prevailing 28 feature sets and the 20 feature sets not considered in previous studies. The results from the new feature set and the existing feature set were closely comparable, underscoring the value of the 20 features. The existing feature set helped ANN and XGBoost to have good model performance compared with the new feature set, which had slight percentage differences. In the Decision Tree, both feature set performances were similar. Interestingly, with the help of a new feature set, LightGBM performed better compared with the existing feature set.

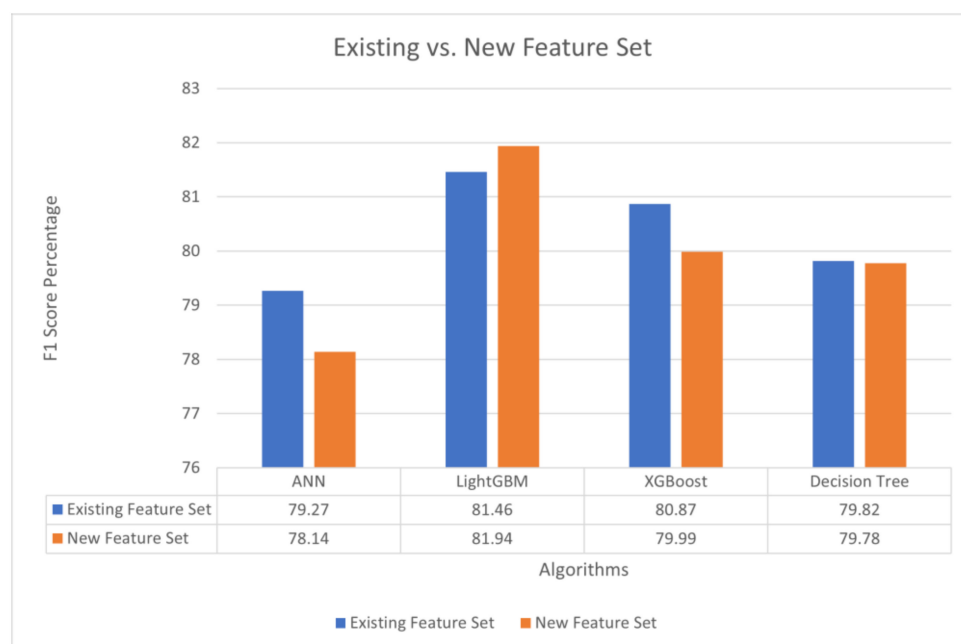


Figure 15. Existing vs. new feature set.

We assessed how using 5, 10, 15, and 20 of the new features affects F1 scores, as shown in Figure 16. All models show consistent performance improvement as the number of features increases from 5 to 20, with the most significant gains occurring up to 15 features. Performance improvements slow down around 20 features, indicating that including more features beyond this number provides little extra benefit.

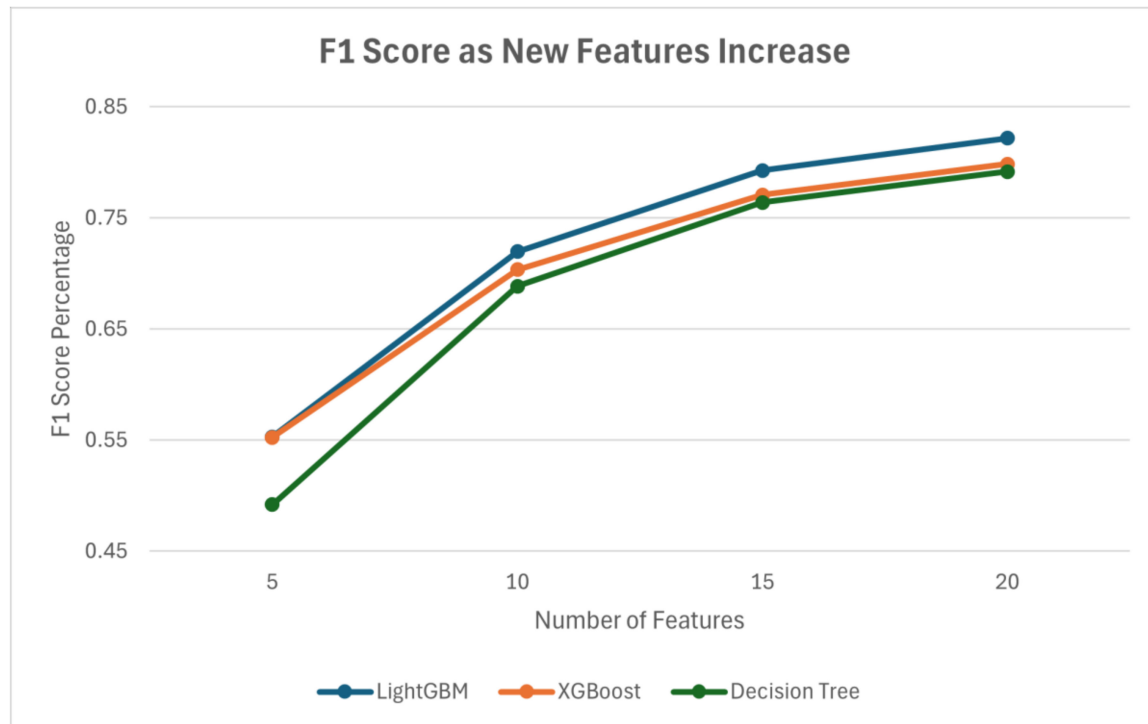


Figure 16. F1 score as new features increase.

The significance of this paper lies in the introduction and demonstration of LeONet, a hybrid architecture that combines LeNet-5 with OreO's Siamese architecture for code clone detection. As per the results obtained, LeONet outperformed all the other models developed in this study, demonstrating its effectiveness in detecting code clones. It showed promising results when compared with state-of-the-art CCD approaches, establishing its competitive edge in the field.

The innovative integration of established architectures, LeNet-5 [18] and OreO's Siamese network [19], into a new hybrid model leverages the strengths of both architectures to address the CCD problem from a fresh perspective. Additionally, new features extracted from AST can provide a more detailed and nuanced representation of code, which enhances the model's capabilities.

The author believes that this innovative application and the introduction of new features contribute valuable insights, improvements to the field and sets the stage for future research, including the evaluation and enhancement of LeONet with code clones in programming languages beyond Java. Thus, this paper's contribution is notable for its performance, the introduction of a new hybrid architecture, and its potential to guide future research in this domain.

5. Limitations of This Study

The proposed approach has not been tested in real-world scenarios or with datasets from other programming languages. Although the controlled environment and dataset used in this study offer valuable insights, they may not fully represent the complexities and variability of practical applications. This limits the understanding of how the approach

performs under diverse and dynamic conditions. While BigCloneBench is a renowned repository in CCD studies [20,33–35], a recent study [62] suggests that it may have limitations and could be considered problematic for evaluating ML approaches. However, its established use provides a common reference point, facilitating comparison with existing methods and ensuring consistency in performance evaluation. It is recognized that further validation with alternative datasets is necessary to comprehensively assess the approach's robustness and applicability. Despite these limitations, LeONet's inference phase is efficient due to its hybrid twin architecture. By operating on low-dimensional, AST-derived vectors, it enables fast forward passes and low memory usage during prediction. These characteristics suggest that LeONet can be practical for real-world deployment in industrial settings, particularly where fast, lightweight clone detection is desired. Future work will specifically address this aspect.

It demonstrates strong theoretical foundations and shows promising results under the conditions of this study. Future work will concentrate on applying and refining the proposed approach in real-world scenarios, as well as exploring alternative datasets and those from other programming languages, to validate its effectiveness and robustness across diverse contexts.

This study did not include direct experimental comparisons with recent Transformer-based approaches such as CodeBERT, GraphCodeBERT, or UniXCoder, which have shown good performance. Due to the defined scope of this work, these models were not incorporated into the current evaluation. We acknowledge this as a limitation of this study. Incorporating such approaches in future work would provide a more comprehensive benchmarking of LeONet's performance relative to recent state-of-the-art approaches and further strengthen the evaluation.

The extracted features are based on counts of specific non-terminal symbols in Java method definitions. While these features effectively capture structural patterns and maintain interpretability, they may lack the semantic richness needed for deeper code understanding. Incorporating more diverse representations, such as code embeddings or graph-based features, is a potential direction for future enhancement. While this study examines the collective impact of increasing feature counts and comparing grouped feature sets, it does not assess the contribution of individual features in isolation. Future work will consider feature importance analysis to identify which specific characteristics most significantly influence clone detection performance.

6. Conclusions and Future Works

This study's primary objective is to detect code clones using DL algorithms. The author collected clone method pairs data from the BigCloneBench Java repository, a renowned repository in the area of CCD. Twenty previously unconsidered features in past studies and other features were extracted during feature extraction. The proposed hybrid DL approach, LeONet, outperformed all other models developed in this study with 98.14% precision, 98.12% recall, and F1 score. It also performed well in comparison to the state-of-the-art CCD approaches. Since the F1 score performances increased as the number of features considered increased, all 48 features utilized in this study were vital in detecting code clones effectively. This revealed how important the 48 features were. It was clear that both the existing feature set and the new feature set yielded comparable F1 score results, highlighting the value of the 20 features.

T2 clone pairs were not detected using the features considered in this study. Apart from literal values, identifiers, and types, there are no differences between T1 and T2 clone pairs. It is recommended to think about extracting features that distinguish them. LeONet consumes a lot of time in the training stage. Developing a strategy that has better

balanced recall and precision ability over all clone types and consumes less training time is recommended. In future work, the author will explore the value of the new features considered in this study, as well as other features, within the context of the proposed model. Exploring a broader feature set remains a promising direction for future work to further enhance clone detection effectiveness. Another potential direction is to evaluate the model using synthetically generated method pairs. For instance, parameterized function pairs with gradually increasing structural complexity, while performing the same computations, can be used to test the limits of the model's clone detection accuracy. Identifying the threshold at which the model begins to fail could offer valuable insights into its generalizability and sensitivity to structural variation.

Author Contributions: Conceptualization, T.V. and K.B.; Methodology, T.V., K.B. and B.T.G.S.K.; Investigation, T.V. and K.B.; Resources, Ashan Induranga; Writing—original draft, T.V., K.B. and A.I.; Writing—review & editing, A.I. and B.T.G.S.K.; Supervision, B.T.G.S.K. and K.K.; Project administration, K.K.; Funding acquisition, K.K. All authors have read and agreed to the published version of the manuscript.

Funding: This research was supported by the Science and Technology Human Resource Development Project, Ministry of Education, Sri Lanka, funded by the Asian Development Bank (Grant No. CRG-R2-SB-1).

Data Availability Statement: The dataset used in this study, BigCloneBench, is publicly available and can be accessed at: <https://github.com/clonebench/BigCloneBench>, accessed on 29 June 2025.

Conflicts of Interest: The authors declare no conflicts of interest.

References

- Kim, M.; Bergman, L.; Lau, T.; Notkin, D. An ethnographic study of copy and paste programming practices in OOPL. In Proceedings of the 2004 International Symposium on Empirical Software Engineering (ISESE 2004), Redondo Beach, CA, USA, 19–20 August 2004; pp. 83–92.
- Ain, Q.U.; Butt, W.H.; Anwar, M.W.; Azam, F.; Maqbool, B. A systematic review on code clone detection. *IEEE Access* **2019**, *7*, 86121–86144. [\[CrossRef\]](#)
- Rattan, D.; Bhatia, R.; Singh, M. Software clone detection: A systematic review. *Inf. Softw. Technol.* **2013**, *55*, 1165–1199. [\[CrossRef\]](#)
- Roy, C.K.; Cordy, J.R. A survey on software clone detection research. *Queen's Sch. Comput. TR* **2007**, *541*, 64–68.
- Monden, A.; Nakae, D.; Kamiya, T.; Sato, S.-I.; Matsumoto, K.-I. Software quality analysis by code clones in industrial legacy software. In Proceedings of the Eighth IEEE Symposium on Software Metrics, Ottawa, ON, Canada, 4–7 June 2002; pp. 87–94.
- Yamashita, A.; Counsell, S. Code smells as system-level indicators of maintainability: An empirical study. *J. Syst. Softw.* **2013**, *86*, 2639–2653. [\[CrossRef\]](#)
- Juergens, E.; Deissenboeck, F.; Hummel, B.; Wagner, S. Do code clones matter? In Proceedings of the 2009 IEEE 31st International Conference on Software Engineering, Vancouver, BC, Canada, 16–24 May 2009; pp. 485–495.
- Barbour, L.; Khomh, F.; Zou, Y. An empirical study of faults in late propagation clone genealogies. *J. Softw. Evol. Process* **2013**, *25*, 1139–1165. [\[CrossRef\]](#)
- Mondal, M.; Roy, C.K.; Schneider, K.A. An empirical study on clone stability. *ACM SIGAPP Appl. Comput. Rev.* **2012**, *12*, 20–36. [\[CrossRef\]](#)
- Mondal, M.; Roy, B.; Roy, C.K.; Schneider, K.A. An empirical study on bug propagation through code cloning. *J. Syst. Softw.* **2019**, *158*, 110407. [\[CrossRef\]](#)
- Zhang, H.; Sakurai, K. A survey of software clone detection from a security perspective. *IEEE Access* **2021**, *9*, 48157–48173. [\[CrossRef\]](#)
- Li, H.; Kwon, H.; Kwon, J.; Lee, H. CLORIFI: Software vulnerability discovery using code clone verification. *Concurr. Comput. Pract. Exp.* **2016**, *28*, 1900–1917. [\[CrossRef\]](#)
- Bellon, S.; Koschke, R.; Antoniol, G.; Krinke, J.; Merlo, E. Comparison and evaluation of clone detection tools. *IEEE Trans. Softw. Eng.* **2007**, *33*, 577–591. [\[CrossRef\]](#)
- Alhazami, E.A.; Sheneamer, A.M. Graph-of-code: Semantic clone detection using graph fingerprints. *IEEE Trans. Softw. Eng.* **2023**, *49*, 3972–3988. [\[CrossRef\]](#)

15. Sheneamer, A.; Roy, S.; Kalita, J. An effective semantic code clone detection framework using pairwise feature fusion. *IEEE Access* **2021**, *9*, 133438–133452. [[CrossRef](#)]
16. Sheneamer, A.; Kalita, J. Semantic clone detection using machine learning. In Proceedings of the 2016 15th IEEE International Conference on Machine Learning and Applications (ICMLA), Anaheim, CA, USA, 18–20 December 2016; pp. 1024–1028.
17. Jo, Y.-B.; Lee, J.; Yoo, C.-J. Two-pass technique for clone detection and type classification using tree-based convolution neural network. *Appl. Sci.* **2021**, *11*, 6613. [[CrossRef](#)]
18. LeCun, Y.; Bottou, L.; Bengio, Y.; Haffner, P. Gradient-based learning applied to document recognition. *Proc. IEEE* **1998**, *86*, 2278–2324. [[CrossRef](#)]
19. Saini, V.; Farmahinifarahani, F.; Lu, Y.; Baldi, P.; Lopes, C.V. OreO: Detection of clones in the twilight zone. In Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Lake Buena Vista, FL, USA, 4–9 November 2018; pp. 354–365.
20. Kaur, M.; Rattan, D. A systematic literature review on the use of machine learning in code clone research. *Comput. Sci. Rev.* **2023**, *47*, 100528. [[CrossRef](#)]
21. Guo, Y.; Liu, Y.; Oerlemans, A.; Lao, S.; Wu, S.; Lew, M.S. Deep learning for visual understanding: A review. *Neurocomputing* **2016**, *187*, 27–48. [[CrossRef](#)]
22. Sajnani, H.; Saini, V.; Svajlenko, J.; Roy, C.K.; Lopes, C.V. SourcererCC: Scaling code clone detection to big-code. In Proceedings of the 38th International Conference on Software Engineering, Austin, TX, USA, 14–22 May 2016; pp. 1157–1168.
23. White, M.; Tufano, M.; Vendome, C.; Poshypanyk, D. Deep learning code fragments for code clone detection. In Proceedings of the 2016 31st IEEE/ACM International Conference on Automated Software Engineering (ASE), Singapore, 3–7 September 2016; pp. 87–98.
24. Jiang, L.; Mishnerghi, G.; Su, Z.; Glondou, S. Deckard: Scalable and accurate tree-based detection of code clones. In Proceedings of the 29th International Conference on Software Engineering (ICSE'07), Minneapolis, MN, USA, 20–26 May 2007; pp. 96–105.
25. Zhao, G.; Huang, J. DeepSim: Deep learning code functional similarity. In Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Lake Buena Vista, FL, USA, 4–9 November 2018; pp. 141–151.
26. Wu, Y.; Zou, D.; Dou, S.; Yang, S.; Yang, W.; Cheng, F.; Hong, L.; Hai, J. SCDetector: Software functional clone detection based on semantic tokens analysis. In Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering, Virtual Event, 21–25 September 2020; pp. 821–833.
27. Zhang, J.; Wang, X.; Zhang, H.; Sun, H.; Wang, K.; Liu, X. A novel neural source code representation based on abstract syntax tree. In Proceedings of the 2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE), Montreal, QC, Canada, 25–31 May 2019; pp. 783–794.
28. Hu, Y.; Zou, D.; Peng, J.; Wu, Y.; Shan, J.; Jin, H. TreeCen: Building tree graph for scalable semantic code clone detection. In Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering, Rochester, MI, USA, 10–14 October 2022; pp. 1–12.
29. Wang, W.; Li, G.; Ma, B.; Xia, X.; Jin, Z. Detecting code clones with graph neural network and flow-augmented abstract syntax tree. In Proceedings of the 2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER), London, ON, Canada, 18–21 February 2020; pp. 261–271.
30. Svajlenko, J.; Islam, J.F.; Keivanloo, I.; Roy, C.K.; Mia, M.M. Towards a big data curated benchmark of inter-project code clones. In Proceedings of the 2014 IEEE International Conference on Software Maintenance and Evolution, Victoria, BC, Canada, 29 September–3 October 2014; pp. 476–480.
31. Svajlenko, J.; Roy, C.K. Evaluating clone detection tools with BigCloneBench. In Proceedings of the 2015 IEEE International Conference on Software Maintenance and Evolution (ICSME), Bremen, Germany, 29 September–1 October 2015; pp. 131–140.
32. Svajlenko, J.; Roy, C.K. BigCloneEval: A clone detection tool evaluation framework with BigCloneBench. In Proceedings of the 2016 IEEE International Conference on Software Maintenance and Evolution (ICSME), Raleigh, NC, USA, 2–7 October 2016; pp. 596–600.
33. Li, L.; Feng, H.; Zhuang, W.; Meng, N.; Ryder, B. CCLearner: A deep learning-based clone detection approach. In Proceedings of the 2017 IEEE International Conference on Software Maintenance and Evolution (ICSME), Shanghai, China, 17–22 September 2017; pp. 249–260.
34. Kim, D.K. Enhancing code clone detection using control flow graphs. *Int. J. Electr. Comput. Eng.* **2019**, *9*, 3287–3296. [[CrossRef](#)]
35. Zeng, J.; Ben, K.; Li, X.; Zhang, X. Fast code clone detection based on weighted recursive autoencoders. *IEEE Access* **2019**, *7*, 125062–125078. [[CrossRef](#)]

36. Falleri, J.-R.; Morandat, F.; Blanc, X.; Martinez, M.; Monperrus, M. Fine-grained and accurate source code differencing. In Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering, Vasteras, Sweden, 15–19 September 2014; pp. 313–324.
37. Eclipse Foundation. Eclipse Java Development Tools (JDT). 2023. Available online: <https://www.eclipse.org/jdt/> (accessed on 2 November 2022).
38. JavaParser—Home. 2022. Available online: <https://javaparser.org/> (accessed on 2 November 2022).
39. Ke, G.; Meng, Q.; Finley, T.; Wang, T.; Chen, W.; Ma, W.; Ye, Q.; Liu, T.Y. LightGBM: A highly efficient gradient boosting decision tree. *Adv. Neural Inf. Process. Syst.* **2017**, *30*, 3146–3154.
40. Chen, T.; Guestrin, C. XGBoost: A scalable tree boosting system. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, CA, USA, 13–17 August 2016; pp. 785–794.
41. de Souza, J.V.A.; Oliveira, L.E.S.E.; Gumiel, Y.B.; Carvalho, D.R.; Moro, C.M.C. Exploiting Siamese neural networks on short text similarity tasks for multiple domains and languages. In Proceedings of the International Conference on Computational Processing of the Portuguese Language, Evora, Portugal, 2–4 March 2020; pp. 357–367.
42. Ondrašovič, M.; Tarábek, P. Siamese visual object tracking: A survey. *IEEE Access* **2021**, *9*, 110149–110172. [[CrossRef](#)]
43. Zhang, W.; Pang, J.; Chen, K.; Loy, C.C. Dense Siamese network for dense unsupervised learning. In Proceedings of the European Conference on Computer Vision, Tel Aviv, Israel, 23–27 October 2022; pp. 464–480.
44. Qiao, Y.; Wu, Y.; Duo, F.; Lin, W.; Yang, J. Siamese neural networks for user identity linkage through web browsing. *IEEE Trans. Neural Netw. Learn. Syst.* **2019**, *31*, 2741–2751. [[CrossRef](#)] [[PubMed](#)]
45. Rao, Y.; Cheng, Y.; Xue, J.; Pu, J.; Wang, Q.; Jin, R.; Wang, Q. FPSiamRPN: Feature pyramid Siamese network with region proposal network for target tracking. *IEEE Access* **2020**, *8*, 176158–176169. [[CrossRef](#)]
46. Yang, B.; Yan, J.; Lei, Z.; Li, S.Z. Convolutional channel features. In Proceedings of the IEEE International Conference on Computer Vision, Santiago, Chile, 7–13 December 2015; pp. 82–90.
47. Li, G.; Yu, Y. Visual saliency detection based on multiscale deep CNN features. *IEEE Trans. Image Process.* **2016**, *25*, 5012–5024. [[CrossRef](#)] [[PubMed](#)]
48. Zaibi, A.; Ladgham, A.; Sakly, A. A lightweight model for traffic sign classification based on enhanced LeNet-5 network. *J. Sens.* **2021**, *2021*, 8870529. [[CrossRef](#)]
49. Kumar, C.R.; Saranya, N.; Priyadharshini, M.; Gilchrist, D. Face recognition using CNN and Siamese network. *Meas. Sens.* **2023**, *27*, 100800. [[CrossRef](#)]
50. Xiong, Y.-J.; Cheng, S.-Y.; Ren, J.-X.; Zhang, Y.-J. Attention-based multiple Siamese networks with primary representation guiding for offline signature verification. *Int. J. Doc. Anal. Recognit. (IJ DAR)* **2024**, *27*, 195–208. [[CrossRef](#)]
51. Zhou, W.; Liu, Y.; Wang, N.; Liang, D.; Peng, B. Efficient Siamese model for visual object tracking with attention-based fusion modules. *Signal Image Video Process.* **2024**, *18*, 1203–1212. [[CrossRef](#)]
52. Han, S.; Nam, H.; Kang, J.; Kim, K.; Cho, S.; Lee, S. Code-Smash: Source-code vulnerability detection using Siamese and multi-level neural architecture. *IEEE Access* **2024**, *12*, 22784–22795. [[CrossRef](#)]
53. Yahya, M.A.; Kim, D.-K. CLCD-I: Cross-language clone detection by using deep learning with InferCode. *Computers* **2023**, *12*, 12. [[CrossRef](#)]
54. Yu, D.; Yang, Q.; Chen, X.; Chen, J.; Xu, Y. Graph-based code semantics learning for efficient semantic code clone detection. *Inf. Softw. Technol.* **2023**, *156*, 107130. [[CrossRef](#)]
55. Tasci, M.; Istanbulu, A.; Kosunalp, S.; Iliev, T.; Stoyanov, I.; Beloev, I. An efficient classification of rice variety with quantized neural networks. *Electronics* **2023**, *12*, 2285. [[CrossRef](#)]
56. An, Y.; Yang, C.; Zhang, S. A lightweight network architecture for traffic sign recognition based on enhanced LeNet-5 network. *Front. Neurosci.* **2024**, *18*, 1431033. [[CrossRef](#)] [[PubMed](#)]
57. Srinivasarao, G.; Rajesh, V.; Saikumar, K.; Baza, M.; Srivastava, G.; Alsabaan, M. Cloud-based LeNet-5 CNN for MRI brain tumor diagnosis and recognition. *Trait. Signal* **2023**, *40*, 223–234. [[CrossRef](#)]
58. Dhayalini, M.; Ponmozhi, B.R.A. Unravelling the mysteries of lung cancer: Harnessing the power of deep learning for detection and classification. In Proceedings of the 2024 International Conference on Recent Advances in Electrical, Electronics, Ubiquitous Communication, and Computational Intelligence (RAEEUCCI), Chennai, India, 16–17 January 2024; pp. 1–6.
59. Sarmah, J.; Saini, M.L.; Kumar, A.; Chasta, V. Performance analysis of deep CNN, YOLO, and LeNet for handwritten digit classification. In Proceedings of the International Conference on Artificial Intelligence on Textile and Apparel, Milan, Italy, 15–17 November 2023; pp. 215–227.
60. Li, B.; Ye, C.; Guan, S.; Zhou, H. Semantic code clone detection via event embedding tree and GAT network. In Proceedings of the 2020 IEEE 20th International Conference on Software Quality, Reliability and Security (QRS), Macau, China, 11–14 December 2020; pp. 382–393.

61. Bisong, E. Introduction to Scikit-learn. In *Building Machine Learning and Deep Learning Models on Google Cloud Platform*; Apress: Berkeley, CA, USA, 2019; pp. 215–229.
62. Krinke, J.; Ragkhitwetsagul, C. BigCloneBench considered harmful for machine learning. In Proceedings of the 2022 IEEE 16th International Workshop on Software Clones (IWSC), Limassol, Cyprus, 2 October 2022; pp. 1–7.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.